Passion Project

Nicola Ouzounov

January 2026

1 Introduction

This is a passion project of mine, where I go out of my way to educate myself. I have been reading about numerical differentiation, approximation etc. Based on the things, that I have read, I am making programs with no real application, other than to get acquainted with numerical methods. My goal is that this knowledge will come in help later down the line in my educational/professional life. With that in mind this document will contain information about the different programs, that I have made, and maybe some comments about the work, put in them.
Contains:

- Euler Approximation Method MATLAB
- RK2 Approximation Method MATLAB
- RK4 Approximation Method MATLAB
- 1-D Simplified Heat Equation MATLAB

2 Euler Approximation Method

This method is used to numerically solve first order differential equations of this type: $y' = f(x, y)$. Using the following property of the derivative we can approximate the original function for very small intervals $x - x_0 = h \to 0$:

$$y'(x) = lim_{x \to x_0} \frac{y(x) - y(x_0)}{x - x_0}$$

$$\Rightarrow y(x) \approx y'(x) * (x - x_0) + y(x_0)$$

$$\Rightarrow y(x) \approx f(x, y) * (x - x_0) + y(x_0)$$

Therefore if we use the given to us function $f(x, y)$ and the initial condition for example $y(0) = y_0$, we can evaluate the next value $y_1, y_2, ...$ etc. Let us take a look at the following example:

$$y' - y = -\frac{1}{2}e^{x/2}sin(5x) + 5e^{x/2}cos(5x)$$

$$\Rightarrow f(x,y) = y - \frac{1}{2}e^{x/2}sin(5x) + 5e^{x/2}cos(5x)$$

$$\Rightarrow y_{n+1} = y_n + f(t_n, y_n)(t_{n+1} - t_n);$$

When put into a MATLAB script, we will make the following boundaries, in order to form a visualization:

- The initial given value is $y(1) = 0$.
- The graph takes place in $x \in [0, 10]$.
- The step size is $h = 0.05$.

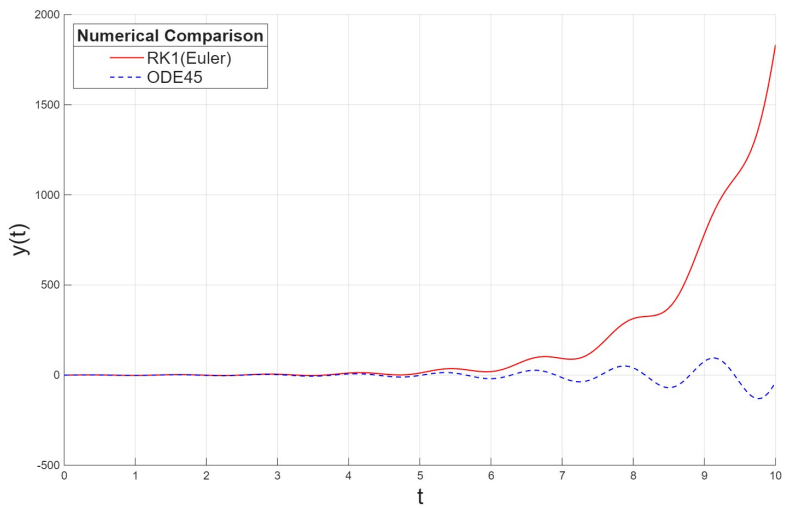


Figure 1: RK1, h = 0.05

As we can see from this image, the method is not converging fast enough. If we make the step size even smaller it converges, of course. To calculate the error, we can simply find the difference between Euler and ODE45. Then the difference is plotted with respect to t:

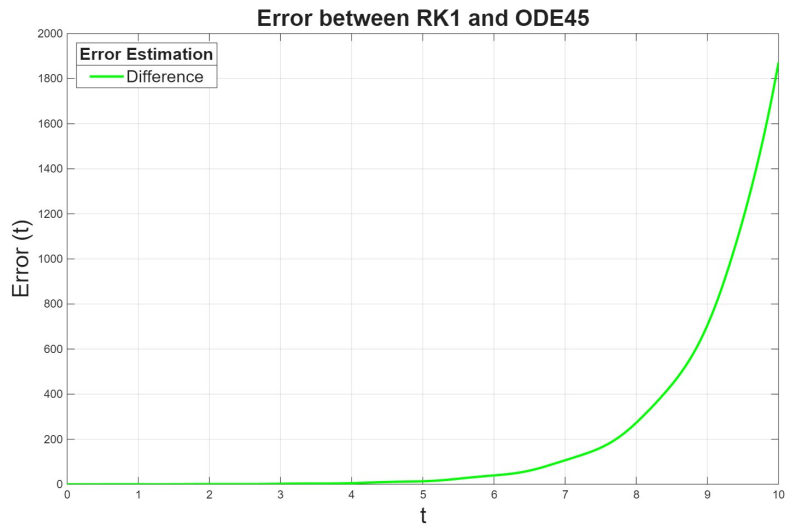


Figure 2: Error Euler

From the original graph of both solutions and from this one it is obvious that the solution is not good at all.

3 RK2 Method

In this section I am introducing the RK2 and RK4 method. We are approximating the same function as in the Euler section, but this time it is compared not to the analytical solution but to the MATLAB function ODE45. The assumptions in place:

- $f(x, y) = y - \frac{1}{2}e^{x/2}sin(5x) + 5e^{x/2}cos(5x)$
- Initial Value Condition $y(0) = 0$
- The step size is $h = 0.05$

Similar to the Euler's Method we will calculate numerically the solution. The difference this time is the accuracy. By Euler's Method the error is $\sim h$. By RK2 the error is $\sim h^2$. The way the RK2 is implemented looks as follows:

$$
\begin{aligned}
A &= f(x_n, y_n) \\
y_{n+1}^{\sim} &= y_n + A * h \\
B &= f(x_n, y_{n+1}^{\sim}) \\
y_{n+1} &= y_n + \frac{A + B}{2} * h
\end{aligned}
$$

We compute an intermediary step for B. This makes the end result converge faster, but also require more calculation step in total.

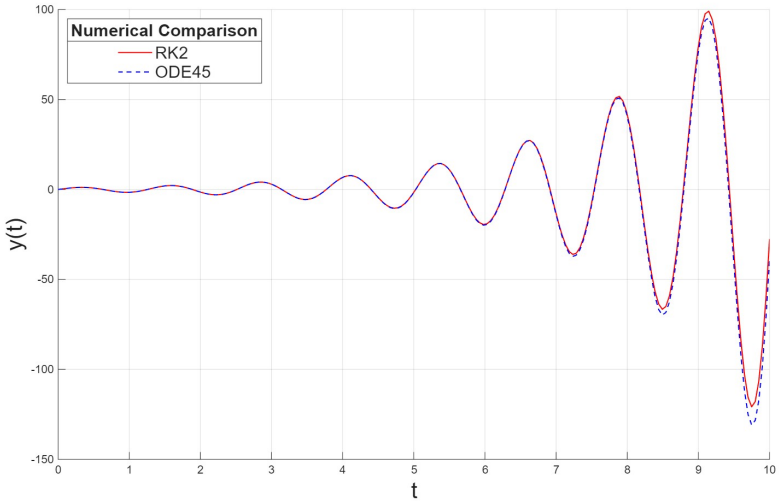


Figure 3: RK2, h = 0.05

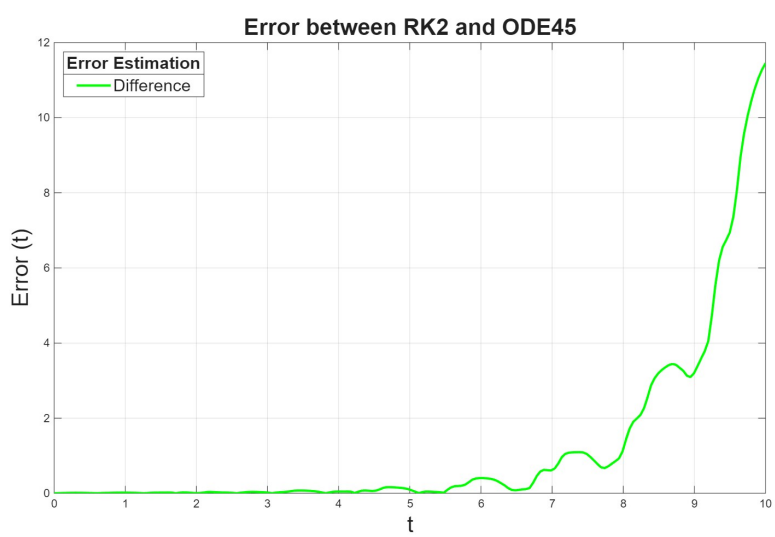


Figure 4: Error RK2

The error in this case, we can see amounts to at most 12. That is to be expected, as the method is supposed to be more accurate.

4 RK4 Method

Similar to RK2, RK4 has 4 coefficients which need to be calculated beforehand. This in turn makes the result more precise, but also require much more power to do so. The calculation is as follows:

$$
\begin{aligned}
A &= f(x_n, y_n) \\
B &= f(x_n + \frac{h}{2}, y_n + \frac{A}{2}h) \\
C &= f(x_n + \frac{h}{2}, y_n + \frac{B}{2}h) \\
D &= f(x_n + h, y_n + Ch) \\
y_{n+1} &= y_n + \frac{A + 2B + 2C + D}{6}h
\end{aligned}
$$

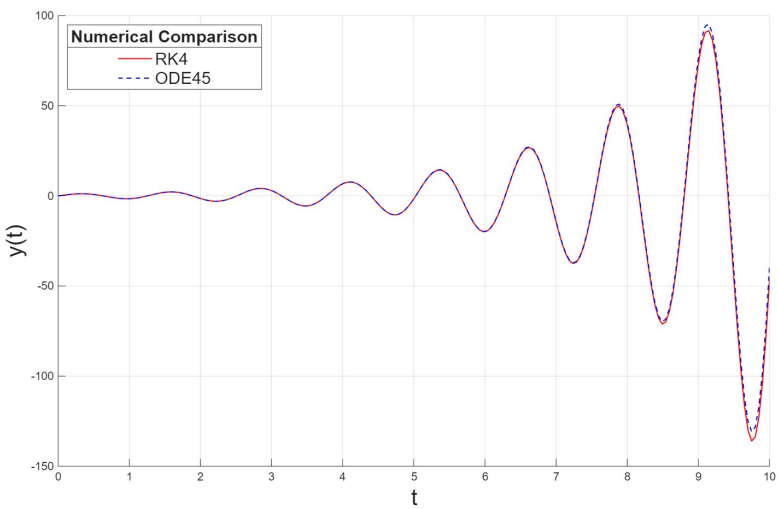


Figure 5: RK4, h = 0.05

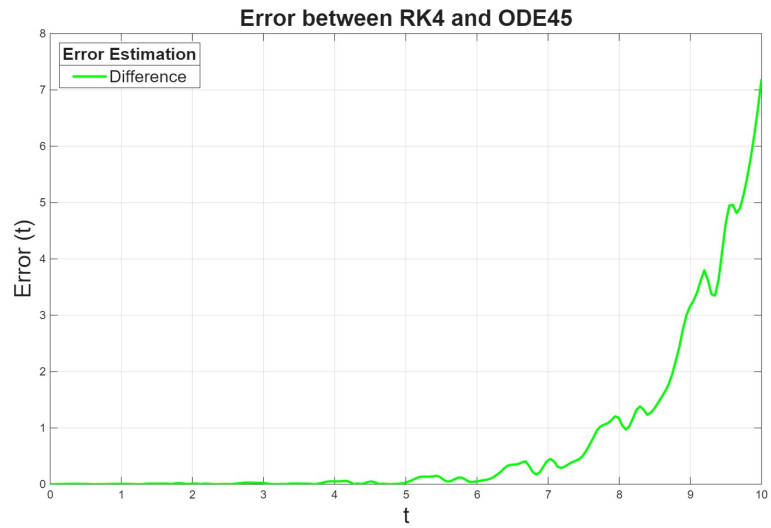


Figure 6: Error RK4

This one is also more accurate than the previous, again expected. In this case, though the difference between the errors of RK2 and RK4 is noticably less than the error difference between RK1 and RK2. As a conclusion we can draw, that in this case the RK4 method is not needed as it requires too much computing power and does not reduce the error by that much. For the ODE in question RK2 is the perfect balance between calculation steps and accuracy.

5 1-D Heat Equation MATLAB

In this section I am showcasing a classic example of how numerical solutions are easier than the analytical ones. The Heat Equation describes the change of temperature with respect to t of a system, in our case in a one dimensional rod with length x.

Assumptions:

- Start and end points of the rod have a fixed temperature.
- We start with a initial temperature distribution.
- The Partial Differential Equation (PDE), being solved is: $\frac{\partial u^2}{\partial^2 x} = \alpha * \frac{\partial u}{\partial t}$.
- The constant α is a physical constant α around $\approx (0, 2)$.

To solve this problem the temperature needs to be determined at all possible points $u(x, t) : \forall x, t$. With the given initial distribution we can numerically

approximate the value of the second derivative with respect to x in all points on the rod:

$$u_{xx}(x,t) = \frac{u_x(x+dx,t) - u_x(x,t)}{dx}$$

$$u_{xx}(x,t) = \frac{u(x+dx,t) - 2u(x,t) + u(x-dx,t)}{dx^2}$$

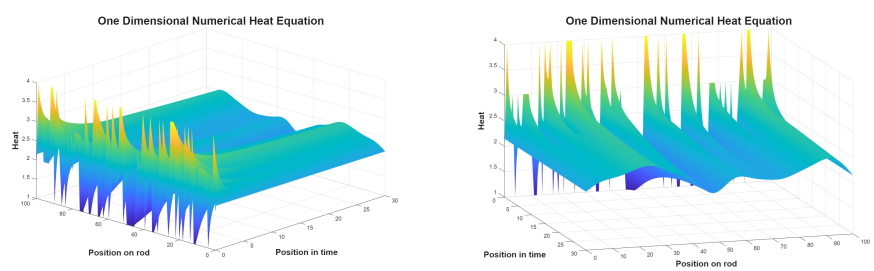
With approximation for the second derivative we can calculate the values for all points on the rod. To find the respective values for the first derivative with respect to time we use the PDE from the start. And to find the value of the temperature at the next moment in time we use the first derivative approximation as follows:

$$u_t(x,t) = \alpha u_{xx}(x,t)$$

$$u_t(x,t) = \frac{u(x,t+dt) - u(x,t)}{dt}$$

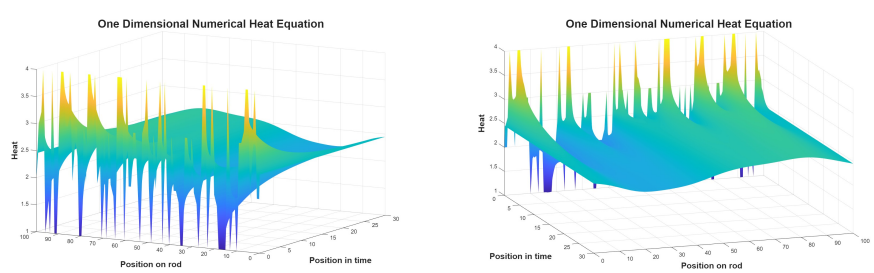
$$\Rightarrow u(x,t+dt) = u_t(x,t)dt + u(x,t)$$

We repeat this algorithm for as long as we want. With the logic ready, all that is left is to implement it in MATLAB. After generating some random noise as input data we get the following results out:



(a) Front (b) Back

Figure 7: Evolution in time for $\alpha = 0.5$



(a) Front (b) Back

Figure 8: Evolution in time for $\alpha = 1.5$

Naturally if we increase the constant, the desired results appear faster and with lower amount of iterations. The problem arises, if we increase the constant too much without readjusting the other input parameters. The result diverges into nonsensical values, which are a result of not respecting the limit for the stability of the solution: $\alpha \leq \frac{dx^2}{2dt}$.

6 References WIP

- mit ode kurs
- MATLAB helpa
- 3b1b youtube videoto za 1dheat
- $https://www.cs.princeton.edu/courses/archive/fall11/cos323/notes/cos323_f11_lecture19_pde2.pdf$